

Aegis

Financial Reconciliation Platform — Project Specification

Version 1.0 (Java Edition) | 25 July 2026 | INTERNAL USE ONLY

Project	Aegis — Financial Reconciliation Platform
Domain	FinTech / Enterprise Finance Operations
Stack	Spring Boot · PostgreSQL · Redis · RabbitMQ · React · Docker
Author	Om Shete DevOm-AI
Status	Draft v1.0 — Architecture & Feature Specification (Java Edition)

1. Problem Statement & Motivation

In any company that processes money — fintechs, payment aggregators, banks, e-commerce platforms, SaaS billing systems — transactions flow through multiple independent systems simultaneously: a payment gateway captures a charge, the bank settles a batch, the internal ledger records a booking, and the ERP posts the revenue. These four records should agree on every rupee. They almost never do — automatically.

What Goes Wrong

- Gateway captures ₹500 but bank settles ₹499.50 (fee deducted differently than expected).
- Payment marked 'success' in internal DB but 'pending' in the gateway due to a webhook drop.
- Refund processed in gateway never reflected in the internal ledger.
- Duplicate transaction IDs from retry storms create phantom credits.
- Currency conversion rate mismatch between capture time and settlement time.
- Bank statement date doesn't match transaction date (float / value dating).

Why Manual Reconciliation Fails at Scale

- 10,000+ transactions/day means every 1% mismatch = 100 rows to investigate manually.
- Finance teams spend 30–40% of month-end close just chasing discrepancies.
- Audit trails are spreadsheet-based — non-reproducible and error-prone.
- No real-time visibility; issues surface days after they occur.

Aegis solves this by automating the full reconciliation lifecycle: ingest → normalize → match → flag → route for review → close → audit.

2. Project Scope

In Scope

- Transaction ingestion from bank statement files (CSV / MT940 / BAI2), payment gateways (REST API), and internal databases.
- Rule-based and fuzzy matching engine with configurable tolerance windows.
- Exception management workflow: auto-flag → assign → resolve → close.
- Multi-entity, multi-currency reconciliation with FX rate integration.
- Scheduled and on-demand reconciliation runs with full job tracking.
- Role-based access: Admin, Finance Analyst, Auditor (read-only).
- Audit log: every match decision, override, and status change is immutable.
- REST API for programmatic access and ERP/accounting integrations.
- Dashboard with KPI metrics, aging exceptions, and trend charts.
- Notification system: email + webhook alerts on exceptions and job completion.

Out of Scope (v1)

- Direct core-banking system integration (planned v2).
- Automated journal entry posting to ERP (planned v2).
- Fraud detection ML models (separate service, API hook available).
- Mobile app.

3. System Architecture

Aegis is a service-oriented modular monolith built on Spring Boot, designed to be extracted into microservices as load requires. All async heavy-lifting goes through RabbitMQ-backed queues and Quartz-scheduled jobs. PostgreSQL is the single source of truth.

Layers & Responsibilities

Layer	Technology	Responsibility
API Gateway	Spring Cloud Gateway + Nginx	Auth, rate-limiting, request routing
Business Logic	Spring Boot (Spring MVC) services	Recon rules, matching logic, exception handling
Task Queue	RabbitMQ + Quartz Scheduler	Async ingestion, scheduled runs, notifications
Data Store	PostgreSQL 15	Transactions, matches, exceptions, audit logs
Cache	Redis (Spring Data Redis)	Job status, hot config, dedup bloom filter
File Storage	S3 / MinIO	Raw ingested files, export reports
Frontend	React + TanStack Query	Dashboard, exception queue, config UI
Infra	Docker + Docker Compose	Local dev; prod: K8s or single VM with Compose

Data Flow (Happy Path)

Step	Stage	Description
1	Ingest	Source file or webhook arrives → stored raw in S3 → job published to RabbitMQ.
2	Parse & Normalize	Format-specific parser converts to canonical Transaction schema. Dedup check via Redis bloom filter.
3	Match Engine	Matching rules run in priority order. Each tx gets a match status: MATCHED / PARTIAL / UNMATCHED.
4	Exception Routing	UNMATCHED and PARTIAL records → Exception queue. Auto-assignment rules apply.
5	Human Review	Finance analyst views exception, investigates, resolves or escalates.
6	Close & Audit	Resolved records marked CLOSED. Immutable audit entry written. Recon run marked complete.

4. Core Features — Detailed Specification

F1 — Multi-Source Transaction Ingestion

Aegis must accept transactions from heterogeneous sources without requiring the source to change its format. A pluggable parser architecture (Java interfaces + Spring-managed strategy beans) handles each format.

- Bank Statement Formats: CSV (custom column mapping), MT940 (SWIFT), BAI2 (US banking).
- Payment Gateway APIs: Razorpay, Stripe, PayU, Cashfree — pulled via scheduled Quartz jobs or webhook listeners exposed through Spring MVC controllers.
- Internal DB Sync: Direct PostgreSQL read via configurable query + schedule, executed through Spring Data JPA repositories.
- Manual Upload: Finance team can upload CSV/Excel via UI with a column-mapping wizard (parsed server-side with Apache POI for Excel).
- Each source has a Source Config record: name, type, credentials (encrypted), cron schedule, parser bean name.
- All raw files stored in S3 with SHA-256 hash. Re-ingestion is idempotent (hash dedup).
- Parse errors isolated per-row — bad rows go to ingestion_errors table, rest proceed.

F2 — Canonical Transaction Schema

All ingested data is normalized to a single internal schema (a JPA @Entity) before any matching occurs. This is the foundation of cross-source reconciliation.

Field	Type	Notes
txn_id	UUID	Internal ID (auto)
external_id	TEXT	Gateway / bank reference ID
source_id	FK → Source	Which source produced this
amount	NUMERIC(18,6)	Always in minor units internally
currency	CHAR(3)	ISO 4217
direction	ENUM	DEBIT CREDIT
txn_date	TIMESTAMPZ	Transaction timestamp (UTC)
value_date	DATE	Bank value / settlement date
description	TEXT	Raw narrative
reference	TEXT	Cheque no / UTR / ARN etc.
status	ENUM	PENDING SETTLED FAILED REFUNDED
metadata	JSONB	Source-specific extra fields
ingested_at	TIMESTAMPZ	When we ingested it

F3 — Rule-Based Matching Engine

The matching engine is the core of the platform. It compares transactions from two or more sources and determines which ones correspond to the same real-world event. Rules are implemented as Java strategy classes behind a common MatchRule interface, executed via a Spring-managed rule pipeline.

3.1 Match Rule Types

Rule Type	How It Works	Config
Exact Match	external_id on both sides is identical	Field names to compare
Amount + Date	Same amount ± tolerance, date within window	Tolerance %, day window
Reference Match	UTR / ARN / cheque number appears in both	Reference field mapping

Rule Type	How It Works	Config
Fuzzy Reference	Levenshtein distance on reference strings	Max edit distance
Aggregate Match	One source has a batch settlement; match sum to individual txns	Group-by field
Manual Override	Human explicitly maps two records	Requires comment

3.2 Match Statuses

- **MATCHED** — Both sides agree within configured tolerance. Auto-closed.
- **PARTIAL** — One side found, amount differs beyond tolerance. Flagged for review.
- **UNMATCHED** — No counterpart found in the other source after all rules run.
- **DUPLICATE** — Same transaction found more than once in the same source.
- **EXCESS** — Record exists on one side with no expectation of a counterpart.

3.3 Matching Run Configuration

- Each Recon Run defines: Source A, Source B, date range, rule set, currency, entity.
- Rules execute in priority order; first rule to match wins (configurable: stop-first or score-all).
- Tolerance: amount (absolute or %) and date ($\pm N$ days) are independently configurable per rule.
- Runs are idempotent — re-running clears previous results for that run_id and re-matches.

F4 — Exception Management Workflow

Every unmatched or partially matched record enters the Exception Queue — a structured workflow to investigate, resolve, and close discrepancies. State transitions are implemented via a Spring State Machine configuration.

State	Description	Who Can Transition
OPEN	Newly flagged, unassigned	System (auto)
ASSIGNED	Assigned to a finance analyst	Admin / System
IN_REVIEW	Analyst is actively investigating	Assigned Analyst
PENDING_INFO	Waiting for external info (bank, gateway)	Analyst
RESOLVED	Root cause identified, adjustment decision made	Analyst
CLOSED	Adjustment posted / write-off approved / matched manually	Admin
ESCALATED	Sent to senior review / management	Analyst
DISPUTED	Formal dispute raised with bank / gateway	Admin

- Each exception has: priority (HIGH / MEDIUM / LOW auto-set by amount threshold), age (days open), notes thread, attachments.
- SLA tracking: exceptions older than a configurable threshold turn red on the dashboard.
- Resolution types: MATCHED_MANUALLY, BANK_ERROR, GATEWAY_ERROR, INTERNAL_ERROR, WRITE_OFF, TIMING_DIFFERENCE.
- All resolution actions require a mandatory comment (min 20 chars) for audit compliance, enforced via Jakarta Bean Validation.

F5 — Multi-Entity & Multi-Currency Support

Enterprise deployments run reconciliation across multiple legal entities and currencies. Aegis handles this natively.

- Entities: Org → Entity hierarchy. Each Source belongs to an Entity. Recon runs scoped per Entity.
- Currency: All amounts stored in original currency + a normalized amount in a base currency (configured per org).
- FX Rates: Daily rate table ingested from configured provider (ECB feed / Fixer.io / manual upload) via a scheduled Quartz job.
- Cross-Currency Match: Optional — match a USD gateway charge against an INR bank debit using spot rate on value_date.

F6 — Scheduled & On-Demand Recon Runs

- Each Source Config has an ingest cron (e.g. pull Razorpay settlements every 2h) managed by Quartz Scheduler.
- Each Recon Profile (pair of sources + rule set) has its own run schedule.
- Manual 'Run Now' available from UI with date-range override.
- Jobs tracked in recon_runs table: status, started_at, completed_at, stats (matched_count, exception_count, etc.).
- Failed jobs: auto-retry with exponential backoff (3 attempts) via Spring Retry. Failure alert sent to admin.

F7 — Audit Trail (Immutable)

Every action in the system produces an immutable audit log entry, persisted via Hibernate Envers-style event listeners. No record is ever hard-deleted — soft deletes only.

Field	Description
event_id	UUID, auto-generated
event_type	INGESTED MATCHED EXCEPTION_CREATED STATUS_CHANGED OVERRIDDEN EXPORTED CONFIG_CHANGED
actor_id	User ID or 'system' for automated actions
entity_ref	Which record was affected (txn_id / exception_id / run_id)
before_state	JSONB snapshot of record before change
after_state	JSONB snapshot of record after change
ip_address	For user-initiated actions
created_at	TIMESTAMPTZ — immutable, never updated

- Audit log is append-only. No UPDATE or DELETE permitted on audit_events table (enforced at DB level via trigger + role restriction).
- Exportable as PDF or CSV for regulator / auditor submission (generated with Apache PDFBox / OpenCSV).
- Retention policy configurable per org (default: 7 years).

F8 — Dashboard & Reporting

The React frontend provides real-time operational visibility, consuming REST endpoints exposed by Spring Boot. Built with TanStack Query for smart caching; charts via Recharts.

Dashboard KPIs (live, auto-refresh every 60s)

- Match Rate % — matched txns / total txns for current period.
- Open Exceptions count — broken down by priority and age bucket.
- Exception Aging — bar chart: 0–1d, 1–3d, 3–7d, 7d+ open.
- Amount at Risk — total rupee value of open exceptions.
- Throughput — txns ingested and matched in last 24h / 7d / 30d.
- Source Health — last successful ingest time per source; red if > threshold.

Reports

- Reconciliation Summary Report: Per run — matched, unmatched, partial, duplicates, total amounts.
- Exception Aging Report: All open exceptions, sortable by amount / age / assignee.
- Analyst Productivity Report: Exceptions resolved per analyst per period.
- Audit Export: Full audit trail for a date range, filtered by entity.
- FX Exposure Report: Unmatched multi-currency items with current FX impact.
- All reports exportable as CSV and PDF.
- Scheduled report delivery via email (Quartz cron-based jobs using Spring Mail).

F9 — Notification & Alerting System

- In-app: Bell icon with unread count. Notification types: new assignment, SLA breach, job failure, run complete.
- Email: Configurable per-user. Digest (daily) or real-time, sent via Spring Mail.
- Webhook: Org-level outbound webhook for integration with Slack, PagerDuty, internal ops systems. Payload: event_type, entity, severity, link.
- Alert Rules: Admin can define: 'if exception amount > ₹50,000 → HIGH priority + immediate email to finance_head@company.com'.

5. REST API Design

All endpoints follow RESTful conventions and are implemented as Spring MVC `@RestController` classes. Auth via JWT (access token 15m + refresh token 7d), issued and validated using Spring Security + JWT. Versioned under `/api/v1/`. All responses wrapped in `{data, meta, errors}` envelope.

Method	Endpoint	Description
POST	<code>/auth/login</code>	Email + password → JWT pair
POST	<code>/auth/refresh</code>	Refresh access token
GET	<code>/sources</code>	List all configured sources
POST	<code>/sources</code>	Create a new source config
POST	<code>/sources/{id}/ingest</code>	Trigger manual ingest for a source
GET	<code>/transactions</code>	List transactions (filter: source, date, status, currency)
GET	<code>/transactions/{id}</code>	Single transaction detail with match info
GET	<code>/recon-runs</code>	List all recon runs
POST	<code>/recon-runs</code>	Trigger a new recon run
GET	<code>/recon-runs/{id}/stats</code>	Run statistics: matched %, counts, amounts
GET	<code>/exceptions</code>	List exceptions (filter: status, priority, assignee, age)
GET	<code>/exceptions/{id}</code>	Exception detail with full history thread
PATCH	<code>/exceptions/{id}/status</code>	Transition exception state
POST	<code>/exceptions/{id}/notes</code>	Add note to exception
POST	<code>/exceptions/{id}/resolve</code>	Resolve with resolution_type + comment
GET	<code>/audit-log</code>	Paginated audit log (filter: event_type, actor, date)
GET	<code>/reports/summary</code>	Reconciliation summary for a run or date range
GET	<code>/reports/exceptions/export</code>	CSV / PDF export of exception report
GET	<code>/dashboard/kpis</code>	Live KPI metrics
POST	<code>/webhooks</code>	Register outbound webhook

Pagination: cursor-based (after=<uuid>) for transaction lists to handle large datasets efficiently, implemented with Spring Data JPA keyset pagination. Rate limiting: 1,000 req/min per API key via a custom Spring Security filter backed by Redis. Bulk endpoints available for batch status updates.

6. Database Schema — Key Tables

Table	Primary Purpose	Key Columns
orgs	Multi-tenancy root	id, name, base_currency, plan
entities	Legal entities within an org	id, org_id, name, tax_id
users	Platform users	id, org_id, email, role, hashed_pw
sources	Ingest source configs	id, entity_id, type, credentials_enc, cron, parser_bean
transactions	Canonical tx store	id, source_id, external_id, amount, currency, direction, txn_date, value_date, status, metadata
recon_profiles	Rule set + source pair config	id, entity_id, source_a_id, source_b_id, rules_json, schedule
recon_runs	Each execution of a profile	id, profile_id, status, started_at, completed_at, matched_count, exception_count
matches	Match results	id, run_id, txn_a_id, txn_b_id, match_type, amount_delta, status
exceptions	Discrepancy records	id, run_id, txn_id, type, status, priority, assigned_to, amount, currency
exception_notes	Threaded comments on exceptions	id, exception_id, author_id, body, created_at
fx_rates	Daily FX rate table	id, base, quote, rate, rate_date, source
audit_events	Immutable audit log	id, event_type, actor_id, entity_ref, before_state, after_state, created_at
notifications	User notification inbox	id, user_id, type, title, body, read_at, created_at
webhooks	Outbound webhook configs	id, org_id, url, secret, events_json, active

- Row-level security (RLS) on all tables: users can only see rows belonging to their org_id, enforced at the Postgres layer beneath the JPA/Hibernate access layer.
- Indexes: (source_id, txn_date), (external_id, source_id) UNIQUE, (exception status, priority), (audit created_at).
- Partitioning: transactions and audit_events partitioned by month for query performance at scale.
- Schema versioning and migrations managed via Flyway, applied automatically on Spring Boot startup.

7. Non-Functional Requirements

NFR	Requirement	Rationale
Performance	Match 100,000 transactions in < 5 min on a standard 4-core server	Daily batch must complete before business hours
Throughput	Ingest API: 500 tx/sec sustained	High-volume gateway webhook support
Availability	99.5% uptime SLA	Finance ops run 24×7
Data Integrity	Zero data loss on consumer crash (manual ack, durable RabbitMQ queues)	Money records must not disappear
Security	AES-256 for source credentials, TLS 1.2+ for all traffic, OWASP Top 10 mitigated	Credential leaks are critical risk
Scalability	Horizontal scaling of Spring Boot worker instances; DB read replicas for report queries	Month-end peak 5× normal load
Auditability	Every state change logged; log retention 7 years	RBI / SOC2 compliance
Observability	Structured JSON logs (Logback), Micrometer metrics, Prometheus + Grafana dashboards	Ops team needs full visibility
Recoverability	RPO: 1 hour, RTO: 4 hours	Daily PG backups + WAL streaming

8. Phased Delivery Roadmap

Phase	Scope	Outcome
Phase 1 — MVP (6 weeks)	Ingestion (CSV + 1 gateway), exact match engine, exception queue, basic dashboard, PostgreSQL, Docker	Can run first real recon manually
Phase 2 — Core (4 weeks)	Rule engine (all match types), multi-source, scheduled runs, email notifications, audit log, export	Finance team can use daily
Phase 3 — Enterprise (4 weeks)	Multi-entity, multi-currency + FX rates, RBAC, API keys, webhook outbound, advanced dashboard	Ready for enterprise clients
Phase 4 — Scale (3 weeks)	DB partitioning, Spring Boot instance autoscale, read replicas, Prometheus + Grafana, SLA tracking	Production-grade at 100k+ tx/day
Phase 5 — Integrations (ongoing)	MT940 / BAI2 parsers, additional gateways, ERP webhook template, scheduled report emails	Ecosystem expansion

9. Technology Stack

Layer	Choice	Why
API Framework	Spring Boot (Spring Web MVC)	Mature enterprise framework, dependency injection, auto-configuration, strong fintech industry adoption
Task Queue	RabbitMQ + Quartz Scheduler	Durable message delivery for financial batch jobs; Quartz for reliable cron-based scheduling
Database	PostgreSQL 15	ACID, JSONB for metadata, partitioning, RLS, triggers for audit
ORM	Spring Data JPA (Hibernate) + Flyway	Type-safe repository pattern, versioned schema migrations
Frontend	React 18 + TanStack Query + Recharts	SPA with smart server-state caching and charting
Auth	Spring Security + JWT + BCrypt	Industry-standard security framework, stateless JWT auth
File Storage	MinIO (dev) / S3 (prod)	Cheap, durable raw file storage
Containers	Docker + Docker Compose	Reproducible environments
Monitoring	Spring Boot Actuator + Micrometer + Prometheus + Grafana	Observability from day 1
Testing	JUnit 5 + Mockito + Testcontainers	Unit + integration testing with real, disposable DB fixtures

10. Key Design Decisions & Tradeoffs

Modular Monolith, not Microservices

At this scale, microservices add ops overhead without benefit. Domain modules (ingestion, matching, exceptions) are cleanly separated as distinct Spring Boot packages/modules but deployed as one service. Extractable when needed.

Idempotent Ingestion via SHA-256 Hash

Every file and every transaction record carries a content hash. Re-uploading the same file or retrying a webhook is safe — duplicates are discarded at the DB level via a UNIQUE constraint on (source_id, external_id).

Rule Engine as JSON Config, not Code

Match rules are stored as JSONB in the DB, not hardcoded. Finance team can adjust tolerances and priorities via UI without a deploy. New rule types require a developer to add a Java executor class implementing the MatchRule interface, but configuration is ops-owned.

Soft Deletes Everywhere

No record is ever physically deleted. Deleted records get deleted_at set (enforced via Hibernate @SQLDelete / @Where annotations). This is non-negotiable for financial systems — you must be able to answer 'what did this look like on date X'.

Amount in Minor Units

All amounts stored as NUMERIC(18,6) in minor units of the currency (e.g. paise, cents). Floating point is never used for money arithmetic. Division/multiplication uses Java's BigDecimal type with an explicit RoundingMode to prevent precision drift.

Async-First Matching

Match runs are never synchronous HTTP responses. POST /recon-runs returns a run_id immediately; the client polls GET /recon-runs/{id}/stats or uses a webhook. This prevents gateway timeouts on large datasets.

11. Security Considerations

- Source Credentials: Gateway API keys and bank SFTP passwords stored encrypted (AES-256-GCM) in DB. Decryption key in environment / Vault. Never logged, never returned via API.
- RBAC: Three roles — Admin (full access), Analyst (own entity + exceptions), Auditor (read-only, no PII) — enforced via Spring Security method-level annotations.
- API Auth: JWT with 15-minute access tokens (JJWT). Refresh token rotation on use. Token revocation via Redis blocklist.
- Row-Level Security: PostgreSQL RLS enforces org isolation at the DB layer — even if application code has a bug, cross-org data leakage is prevented.
- Input Validation: All inputs validated via Jakarta Bean Validation (`@Valid`, `@NotNull`, custom validators). File uploads: type checked (magic bytes, not extension), size-limited.
- Audit Immutability: `audit_events` table has a DB-level trigger that blocks UPDATE and DELETE. The application's DB user has no DELETE permission on this table.
- Rate Limiting: Per-API-key and per-IP limits via a Redis sliding-window filter registered in the Spring Security filter chain.
- Secrets Management: No secrets in code or Docker images. All via environment variables / Spring Config properties; prod via Vault or AWS SSM.

12. Open Questions / Decisions Pending

#	Question	Options	Owner
1	Which payment gateways to support in MVP?	Razorpay only vs Razorpay + Stripe	Om / Client
2	MT940 / BAI2 parser — build or use library?	Java MT940/BAI2 parsing library vs custom parser	Om
3	FX rate provider?	ECB free feed vs Fixer.io paid vs manual upload	Client
4	Deployment target for v1?	Single VPS + Docker Compose vs managed K8s	Client
5	Compliance requirement level?	Internal only vs SOC2 / RBI audit readiness	Client
6	Multi-tenancy in v1?	Single org (simpler) vs multi-org SaaS from day 1	Om / Client

13. Glossary

Term	Definition
Reconciliation	The process of matching transaction records from two or more sources to verify they agree
Exception	A transaction that could not be automatically matched and requires human investigation
Source	Any system that produces transaction data: bank, gateway, internal DB
Match Rule	A configured criterion used to determine if two transactions represent the same real-world event
Recon Run	One execution of a reconciliation profile over a defined date range
Recon Profile	A saved configuration: which two sources to compare, which rules to apply, what schedule to run on
MT940	SWIFT standard format for bank statement files
BAI2	US banking standard format for cash management files
UTR	Unique Transaction Reference — used in NEFT / RTGS in India
ARN	Acquirer Reference Number — used in card payment networks
Minor Units	Smallest denomination of a currency (paise for INR, cents for USD)
Value Date	The date a transaction actually affects a bank balance (may differ from transaction date)
Float	The difference in timing between when a payment is initiated and when it settles